

机器学习与数据挖掘/理论作业3

姓名	学号
汪宣彤	22336221

1. Transformer为何使用多头注意力机制

◦ 提高表示的丰富性

不同的注意力头可以学习到不同的特征模式，多头注意力将这些模式组合起来，生成更加丰富和多样化的特征表示，使得模型能够更好地理解复杂的输入结构

◦ 多角度捕捉信息

多头注意力机制允许模型从不同的角度理解序列中的关系，每个注意力头可以关注输入序列中的不同部分，这使得模型能够同时关注局部和全局的上下文信息

◦ 避免信息丢失

单一注意力头可能会集中在输入的某些特定特征上，而忽略其他重要的信息。通过使用多个注意力头，Transformer可以在不同的头之间捕捉更全面的信息，从而避免单头注意力可能带来的信息丢失问题

2. Q、K、V分别是什么，怎么得到的，代表什么含义

Q (Query) 查询向量

- **来源**：输入序列中的每个词向量经过一个线性变换得到
- **作用**：Q用于表示查询，即当前要关注的词向量
- **含义**：Q代表要提问的信息，即该词想要知道的上下文关系

K (Key) 键向量

- **来源**：输入序列的词向量经过另一个线性变换得到
- **作用**：K用于与Q进行点积计算，表示当前词与哪些其他词相关
- **含义**：K相当于所有词的信息，它被用来与Q匹配，以判断当前词应当关注哪些词

V (Value) 值向量

- **来源**：输入序列的词向量经过另一个线性变换得到
- **作用**：Q和K通过注意力机制计算出需要关注的权重，然后将权重作用到V上，输出最终的结果
- **含义**：V表示包含的信息，即模型最终根据注意力分配提取的信息

3. 为什么在进行softmax之前需要对attention除以d的平方根

- **softmax对大数值的敏感性**

- softmax函数的输出对输入值的范围非常敏感
- 如果点积的值过大，那么在经过softmax时，较大的输入值会导致它们在输出中占据几乎全部的权重，导致模型只关注少数几个位置，忽略其他位置的信息
- 这会降低模型的表达能力，使得模型训练效果变差

- **点积值随维度增大而增大**

- 当输入向量的维度较高时，Query和Key的点积值的期望会随着维度的增大而增大
- 对于两个随机生成的长度为 d_k 的向量，它们的点积的期望值是与 d_k 成正比
- 如果不对点积进行缩放，随着 d_k 的增加，点积值可能会非常大。大的点积值输入到softmax函数后，softmax的输出分布会变得非常尖锐，也就是大部分权重集中在少数位置，其他位置几乎被忽略

- **缩放后稳定注意力机制的效果**

- 为了防止这种现象发生，Transformer引入了缩放因子 $\frac{1}{\sqrt{d_k}}$ ，这使得点积值被适当缩小到一个合理范围
- 经过缩放后，点积值不会过大，从而使得softmax输出的分布更加平滑，权重分布更加合理，使得模型能够更稳定地关注不同位置的词语之间的关系

4. Encoder和Decoder是如何进行交互的

它们通过**多头注意力机制**来进行交互，具体过程如下：

1. Encoder自注意力

- 在Encoder中，每个词对其他所有输入词进行自注意力计算，通过多头注意力机制提取句子中不同词之间的依赖关系，生成新的表示
- 这一过程完全基于输入序列自身，输入序列的所有词都能相互“看到”对方

2. Encoder生成上下文表示

- Encoder通过多个自注意力层的堆叠，逐步抽取输入序列的全局特征，生成一组上下文表示向量
- 这些向量被传递给Decoder，用于后续解码过程

3. Decoder自注意力

- 在Decoder中，首先对已经生成的部分目标序列进行自注意力计算
- 这一步和Encoder的自注意力机制类似，不同的是，Decoder的自注意力机制是“掩蔽的”

4. Encoder-Decoder交互

- 交互的关键步骤发生在第二个注意力层，也就是**编码-解码注意力机制**。在这一层，Decoder中的每个词会与Encoder输出的上下文表示（即编码向量）进行注意力计算
 - Decoder的Query来自目标序列的自注意力结果（前一步生成的词的表示）
 - Encoder的Key和Value来自Encoder的输出向量（输入序列的编码表示）
- 通过多头注意力机制，Decoder的每个词根据输入序列中不同位置的信息来生成新词，这样使得Decoder能够根据整个输入句子的全局信息来解码每个词

5. 最终输出

- 经过自注意力、编码-解码注意力和前馈神经网络，Decoder逐步生成目标序列的每个词。在序列到序列任务中，这个过程一直进行到生成完整的目标序列

5. 为什么Decoder自注意力需要进行 mask

防止信息泄露和未来信息干扰

- 很多任务中，Decoder是一个逐步生成序列的过程。模型在生成目标序列的每个词时，应该只依赖于已经生成的词，而不能“偷看”未来的词
- 为了保证这一点，必须对Decoder中的自注意力机制进行掩蔽，即只允许当前词看到它之前生成的词，屏蔽掉它未来的词
- 如果不进行掩蔽，Decoder中的每个词就能访问到整个目标序列的信息，包括还未生成的词。这会导致模型在生成时依赖于未来词语的信息，从而违背了生成式模型的因果性要求，并可能使模型的训练失效或输出不合理的结果
- 掩蔽保证了序列生成任务的因果性，使得模型逐步生成合理的输出